# JavaScript Từ Zero Đến Hero

Tài liệu gốc kèm thực hành bổ sung · 29/10/2026

## Mục lục


- Nhập Môn JavaScript: JavaScript là gì?
- Nhập Môn JavaScript: Biến, kiểu dữ liệu & toán tử
- Nhập Môn JavaScript: Hàm trong JavaScript
- DOM & Xử Lý Sự Kiện: Tổng quan về DOM
- DOM & Xử Lý Sự Kiện: Xử lý sự kiện click, submit
- DOM & Xử Lý Sự Kiện: Dự án: To-do List
- JavaScript Hiện Đại (ES6+): Arrow function & Destructuring
- JavaScript Hiện Đại (ES6+): Promise & Async/Await
- JavaScript Hiện Đại (ES6+): Module & Fetch API

CHƯƠNG 1 · BÀI 1

# 1.1. JavaScript là gì?

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu JavaScript là gì và dùng để làm gì
> - Biết cách chạy JavaScript trong trình duyệt
> - Viết được chương trình JavaScript đầu tiên

## 1. JavaScript là gì

JavaScript là ngôn ngữ lập trình chạy trực tiếp trong trình duyệt, dùng để tạo tương tác cho trang web. Một trang web gồm ba lớp: HTML mô tả cấu trúc, CSS định dạng giao diện và JavaScript xử lý hành vi. Ngày nay JavaScript còn chạy được trên máy chủ nhờ Node.js.

## 2. Cách chạy JavaScript

Có hai cách nhúng mã vào trang: viết giữa cặp thẻ `script` hoặc liên kết file ngoài bằng thuộc tính `src`. Nên đặt thẻ script ở cuối `body` hoặc dùng thuộc tính `defer` để trang tải xong rồi mới chạy. Có thể thử nhanh mã trong Console của trình duyệt (phím F12).

```
<script src="app.js" defer></script>
```

## 3. Chương trình đầu tiên

```
console.log("Xin chào JavaScript!");
alert("Chào mừng bạn đến với StudyOnline");
```

`console.log` in ra Console, dùng khi gỡ lỗi; `alert` hiện hộp thoại. Chú thích một dòng dùng `//`, nhiều dòng dùng `/* ... */`. Câu lệnh nên kết thúc bằng dấu chấm phẩy.

## Tóm tắt

- JavaScript xử lý hành vi trang web, chạy trong trình duyệt và cả trên máy chủ (Node.js).
- Nhúng mã bằng thẻ `script`; dùng `defer` hoặc đặt cuối body.
- `console.log` dùng để kiểm tra và gỡ lỗi.

## Câu hỏi ôn tập và bài tập vận dụng

1. Tạo file index.html liên kết file app.js và in ra tên của bạn bằng console.log.
2. Dùng alert hiển thị một lời chào.
3. Liệt kê ba vai trò của HTML, CSS và JavaScript trong một trang web.

# JavaScript là gì? — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

## Tình huống, ví dụ và tự kiểm tra

**Tình huống:** "Em mở file JS mà không có gì hiện ra." Mã JavaScript cần môi trường thực thi. Node không có document của trình duyệt; trình duyệt chạy script trong trang và hiển thị console ở công cụ phát triển.

```
console.log("Hello StudyOnline");
console.assert(2 + 3 === 5);
```

Trong HTML, dùng `<script src="app.js" defer></script>` để tải file ngoài và chạy sau khi phân tích HTML. Mở Network để xem app.js có tải thành công không; đường dẫn tương đối phụ thuộc vị trí trang. Phân biệt lỗi tải file với lỗi cú pháp và lỗi logic.

**Tự luyện:** in một dòng khi trang mở và một dòng khi nhấn nút. **Hướng giải:** dòng đầu chứng minh script chạy; dòng sau chứng minh sự kiện đã gắn đúng. Nếu có lỗi trước đoạn gắn sự kiện, cần sửa lỗi đó trước.

## Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

CHƯƠNG 1 · BÀI 2

# 1.2. Biến, kiểu dữ liệu & toán tử

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

### Mục tiêu bài học

- Khai báo biến bằng var, let, const và hiểu sự khác nhau
- Nắm các kiểu dữ liệu của JavaScript
- Sử dụng toán tử và hiểu sự khác nhau giữa == và ===

## 1. Khai báo biến

ES6 bổ sung `let` và `const`. `const` dùng để khai báo biến không đổi (không gán lại được); `let` dùng cho biến có thể thay đổi và có phạm vi khối. `var` là cách cũ, có phạm vi hàm và dễ gây lỗi hoisting. Nên ưu tiên `const`, chỉ dùng `let` khi cần gán lại.

```
const PI = 3.14;
let dem = 0;
dem = dem + 1;
```

## 2. Kiểu dữ liệu

JavaScript có các kiểu nguyên thuỷ: number, string, boolean, null, undefined, symbol, bigint. Ngoài ra có object (gồm mảng và hàm). Một điểm gây nhầm lẫn nổi tiếng: `typeof null` trả về "object" (lỗi lịch sử của ngôn ngữ, vẫn được giữ để tương thích ngược).

```
console.log(typeof 42);        // "number"
console.log(typeof "abc");     // "string"
console.log(typeof undefined); // "undefined"
```

## 3. Toán tử

Toán tử số học giống nhiều ngôn ngữ: `+ - * / % **`. Toán tử + với chuỗi sẽ nối chuỗi. JavaScript có hai kiểu so sánh bằng: == so sánh sau khi ép kiểu, còn === so sánh cả giá trị lẫn kiểu. Nên luôn dùng === và !==.

```
console.log(5 == "5");   // true
console.log(5 === "5");  // false
console.log("5" + 3);    // "53"
console.log("5" - 3);    // 2
```

## 4. Giá trị đúng sai (truthy, falsy)

Các giá trị falsy gồm `false`, `0`, `""`, `null`, `undefined`, `NaN`; mọi giá trị khác là truthy. Toán tử logic là `&&`, `||`, `!`.

## Tóm tắt

- Ưu tiên `const`, dùng `let` khi cần gán lại, tránh `var`.
- Kiểu nguyên thuỷ: number, string, boolean, null, undefined, symbol, bigint.
- Dùng `===` thay cho `==` để tránh ép kiểu bất ngờ.

## Câu hỏi ôn tập và bài tập vận dụng

1. Khai báo các biến lưu tên, tuổi, trạng thái học viên và in kiểu của từng biến bằng typeof.
2. Dự đoán kết quả của `"10" * 2` và `"10" + 2`, sau đó chạy để kiểm tra.
3. Giải thích sự khác nhau giữa `==` và `===` kèm ví dụ.

# Biến, kiểu dữ liệu & toán tử — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

## Tình huống, ví dụ và tự kiểm tra

**Hỏi thực tế:** "const rồi sao vẫn push được?" const giữ binding không được gán lại, không đóng băng đối tượng. let cho phép gán lại; ưu tiên const khi không cần gán lại biến.

```
const items = [];
items.push("A");
console.assert(items.length === 1);
console.assert("2" + 3 === "23");
console.assert(Number("2") + 3 === 5);
console.assert(("2" == 2) === true);
console.assert(("2" === 2) === false);
```

Toán tử + có chuỗi sẽ nối chuỗi trong trường hợp trên. `==` có ép kiểu; `===` kiểm tra nghiêm ngặt, nên dùng khi không chủ đích ép kiểu. `Number("")` cho 0 nên kiểm tra rỗng trước khi đổi số.

**Tự luyện:** kiểm tra chuỗi khoảng trắng và `"abc"`. **Hướng giải:** loại input rỗng; sau chuyển đổi dùng `Number.isFinite`. Không dùng kiểm tra truthy để loại số 0 nếu 0 là dữ liệu hợp lệ.

## Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

CHƯƠNG 1 · BÀI 3

# 1.3. Hàm trong JavaScript

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Định nghĩa hàm bằng function declaration, function expression và arrow function
> - Sử dụng tham số, giá trị mặc định và giá trị trả về
> - Hiểu hàm là giá trị có thể truyền như biến

## 1. Khai báo hàm

Hàm là khối lệnh có tên để tái sử dụng. Function declaration được nâng lên đầu phạm vi (hoisting) nên có thể gọi trước khi khai báo.

```
function cong(a, b) {
  return a + b;
}
console.log(cong(2, 3)); // 5
```

## 2. Function expression và arrow function

Function expression gán hàm cho một biến. Arrow function là cú pháp ngắn gọn: `const add = (a, b) => a + b;`. Với một tham số có thể bỏ ngoặc: `x => x * 2`. Arrow function không có `this` riêng, nó kế thừa `this` từ ngữ cảnh bao quanh.

```
const nhan = function (a, b) { return a * b; };
const binhPhuong = (x) => x * x;
```

## 3. Tham số mặc định và rest

Tham số có thể có giá trị mặc định: `function chao(ten = "bạn") {...}`. Cú pháp rest `...args` gom các đối số còn lại thành một mảng.

```
function tong(...so) {
  return so.reduce((t, x) => t + x, 0);
```

```
}
console.log(tong(1, 2, 3, 4)); // 10
```

## 4. Hàm là giá trị

Hàm có thể được gán vào biến, truyền làm đối số cho hàm khác (callback) hoặc được trả về từ hàm. Đây là nền tảng của xử lý sự kiện và lập trình bất đồng bộ.

## Tóm tắt

- Ba cách viết hàm: declaration, expression, arrow.
- Tham số mặc định và rest giúp hàm linh hoạt hơn.
- Hàm là giá trị nên có thể làm callback.

## Câu hỏi ôn tập và bài tập vận dụng

1. Viết hàm kiểm tra một số có phải số chẵn hay không, bằng cả function thường và arrow function.
2. Viết hàm `tong(...so)` tính tổng số lượng đối số bất kỳ.
3. Viết hàm nhận một mảng và một callback rồi áp dụng callback cho từng phần tử.

# Hàm trong JavaScript — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

## Tình huống, ví dụ và tự kiểm tra

**Tình huống:** chương trình in đúng nhưng biến nhận kết quả lại undefined. Hàm phải return giá trị muốn truyền cho nơi gọi; console.log chỉ ghi ra console.

```
function total(values) {
  return values.reduce((sum, value) => sum + value, 0);
}
console.assert(total([2, 3]) === 5);
console.assert(total([]) === 0);
const add = (a, b) => a + b;
console.assert(add(1, 2) === 3);
```

Hợp đồng hàm cần nêu mảng có chứa chuỗi không, cách xử lý dữ liệu sai và có sửa input hay không. Callback là hàm được truyền vào hàm khác; truyền `handler` khác với gọi ngay `handler()`.

**Tự luyện:** viết hàm tính giá sau giảm, kiểm tra phần trăm trong 0..100. **Hướng giải:** tách kiểm tra dữ liệu khỏi thao tác giao diện, thử 0%, 100% và giá trị ngoài miền.

## Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

CHƯƠNG 2 · BÀI 1

# 2.1. Tổng quan về DOM

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu DOM là gì và mô hình cây phần tử
> - Chọn phần tử bằng querySelector và các phương thức khác
> - Thay đổi nội dung, thuộc tính và kiểu của phần tử

## 1. DOM là gì

DOM (Document Object Model) là biểu diễn cây của trang HTML. Mỗi thẻ là một nút trong cây; JavaScript dùng đối tượng `document` để đọc và thay đổi trang mà không cần tải lại.

## 2. Chọn phần tử

Để chọn một phần tử theo id, dùng `document.getElementById("myId")`. Các phương thức khác: `querySelector` (trả về phần tử đầu tiên khớp selector CSS), `querySelectorAll` (trả về danh sách mọi phần tử khớp) và `getElementsByClassName`.

```
const tieuDe = document.querySelector("h1");
const cacMuc = document.querySelectorAll(".item");
```

## 3. Thay đổi nội dung và kiểu

Có thể thay đổi nội dung bằng thuộc tính `textContent` (chỉ văn bản, an toàn) hoặc `innerHTML` (chèn HTML, cần cẩn thận với dữ liệu người dùng vì có nguy cơ tấn công XSS). Thuộc tính `style` và `classList` dùng để đổi giao diện.

```
tieuDe.textContent = "Xin chào DOM";
tieuDe.style.color = "tomato";
tieuDe.classList.add("noi-bat");
```

## 4. Tạo và xoá phần tử

`document.createElement("li")` tạo phần tử mới; `cha.appendChild(con)` thêm vào cây; `phanTu.remove()` xoá khỏi cây.

```
const li = document.createElement("li");
li.textContent = "Mục mới";
document.querySelector("ul").appendChild(li);
```

## Tóm tắt

- DOM là cây phần tử của trang, truy cập qua `document`.
- Dùng `querySelector` / `querySelectorAll` để chọn phần tử.
- `textContent` an toàn hơn `innerHTML` khi hiển thị dữ liệu người dùng.

## Câu hỏi ôn tập và bài tập vận dụng

1. Đổi nội dung và màu của một tiêu đề khi trang tải xong.
2. Tạo danh sách 5 mục bằng vòng lặp và createElement.
3. Giải thích vì sao không nên gán dữ liệu người dùng vào innerHTML.

# Tổng quan về DOM — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

## Tình huống, ví dụ và tự kiểm tra

**Hỏi thực tế:** "getElementById('btn') ra null nên nút không hoạt động." Kiểm tra phần tử có đúng id không, có được render không và script có chạy trước khi HTML được phân tích không. Không kết luận một nguyên nhân khi chưa có HTML và vị trí script.

```
const button = document.getElementById("btn");
if (!button) {
  console.error("Không tìm thấy #btn; kiểm tra HTML và thời điểm chạy");
} else {
  button.textContent = "Sẵn sàng";
}
```

`textContent` phù hợp để hiển thị văn bản người dùng nhập. Không ghép dữ liệu chưa tin cậy vào innerHTML. Dùng defer cho script ngoài hoặc gắn sau DOMContentLoaded khi cần.

**Tự luyện:** đổi id trong HTML rồi quan sát thông báo. **Hướng giải:** sau khi khớp id và thời điểm, đối tượng không còn null; tránh thêm setTimeout tùy ý để che lỗi thứ tự.

## Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

CHƯƠNG 2 · BÀI 2

# 2.2. Xử lý sự kiện click, submit

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Gắn sự kiện bằng addEventListener
> - Xử lý các sự kiện click, input và submit
> - Hiểu đối tượng event và hành vi mặc định

## 1. Sự kiện và trình xử lý

Sự kiện là tác động của người dùng hoặc trình duyệt như nhấp chuột, gõ phím, gửi form. Dùng `addEventListener("click", handler)` để gắn hàm xử lý vào phần tử. Có thể gắn nhiều trình xử lý cho cùng một sự kiện.

```
const nut = document.querySelector("#nut");
nut.addEventListener("click", () => {
  console.log("Đã bấm");
});
```

## 2. Đối tượng event

Hàm xử lý nhận một đối số là đối tượng `event`. `event.target` là phần tử gây ra sự kiện; `event.preventDefault()` chặn hành vi mặc định, ví dụ không cho form tải lại trang khi submit.

```
form.addEventListener("submit", (e) => {
  e.preventDefault();
  console.log(form.elements.ten.value);
});
```

## 3. Một số sự kiện thường dùng

`click`, `dblclick`, `input` (mỗi lần giá trị ô nhập thay đổi), `change`, `keydown`, `submit`, `mouseover`, `DOMContentLoaded` (khi HTML đã tải xong).

## 4. Lan truyền sự kiện

Mô hình sự kiện gồm giai đoạn capturing (từ ngoài vào) và bubbling (từ trong ra ngoài). Mặc định `addEventListener` lắng nghe ở giai đoạn bubbling. Nhờ đó có thể gắn một trình xử lý ở phần tử cha để xử lý cho nhiều phần tử con (event delegation).

## Tóm tắt

- `addEventListener(tên, hàm)` gắn trình xử lý sự kiện.
- `preventDefault()` chặn hành vi mặc định của form hoặc liên kết.
- Sự kiện lan truyền từ phần tử con lên cha (bubbling), có thể tận dụng để uỷ quyền sự kiện.

## Câu hỏi ôn tập và bài tập vận dụng

1. Làm nút đếm: mỗi lần bấm tăng số đếm và hiển thị lên trang.
2. Làm form đăng ký kiểm tra tên không để trống trước khi gửi.
3. Dùng event delegation xử lý click cho mọi mục của một danh sách.

# Xử lý sự kiện click, submit — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

## Tình huống, ví dụ và tự kiểm tra

**Tình huống:** bấm nút trong form làm tải lại trang. Nút mặc định trong form có thể submit; với hành động riêng dùng `type="button"`, với gửi form xử lý sự kiện submit và preventDefault khi ứng dụng tự gửi request.

```
const form = document.querySelector("form");
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const name = new FormData(form).get("name");
  console.log(String(name ?? "").trim());
});
```

**Tự luyện:** nhấn Enter trong input và nhấn nút gửi, mỗi hành động chỉ được xử lý một lần. **Hướng giải:** nghe submit trên form thay vì chỉ click trên nút; tránh gắn lại handler mỗi lần render. Khi gửi bất đồng bộ, khóa nút trong lúc chờ và mở lại ở finally; backend vẫn phải kiểm soát thao tác trùng.

## Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

CHƯƠNG 2 · BÀI 3

# 2.3. Dự án: To-do List

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Kết hợp DOM, sự kiện và mảng để xây dựng ứng dụng To-do List
> - Thêm, đánh dấu hoàn thành và xoá công việc
> - Lưu dữ liệu bằng localStorage

## 1. Yêu cầu

Ứng dụng cho phép nhập công việc, thêm vào danh sách, bấm vào công việc để đánh dấu hoàn thành, xoá công việc và giữ lại danh sách khi tải lại trang.

## 2. Cấu trúc HTML

Một ô nhập `input`, một nút thêm và một danh sách `ul`. Mỗi công việc là một thẻ `li` chứa nội dung và nút xoá.

## 3. Cài đặt mẫu

Tách dữ liệu (mảng `todos`) khỏi giao diện: mỗi khi dữ liệu thay đổi thì gọi hàm `render()` để vẽ lại danh sách.

```
let todos = JSON.parse(localStorage.getItem("todos") || "[]");

function luu() {
  localStorage.setItem("todos", JSON.stringify(todos));
}

function render() {
  const ul = document.querySelector("#ds");
  ul.innerHTML = "";
  todos.forEach((t, i) => {
    const li = document.createElement("li");
    li.textContent = t.ten;
    li.className = t.xong ? "xong" : "";
    li.addEventListener("click", () => { t.xong = !t.xong; luu(); render(); });
```

```
    ul.appendChild(li);
  });
}

document.querySelector("#them").addEventListener("click", () => {
  const o = document.querySelector("#ten");
  if (o.value.trim()) { todos.push({ ten: o.value.trim(), xong: false }); o.value = "";
luu(); render(); }
});
render();
```

## 4. Gợi ý mở rộng

Thêm bộ lọc Tất cả, Đang làm, Đã xong; sửa tên công việc; hiển thị số công việc còn lại; thêm phím tắt Enter để thêm công việc.

## Tóm tắt

- Giữ dữ liệu trong mảng, dùng hàm render để cập nhật giao diện.
- localStorage lưu chuỗi, dùng `JSON.stringify` và `JSON.parse` để lưu đối tượng.
- Xử lý sự kiện trên từng phần tử hoặc dùng event delegation.

## Câu hỏi ôn tập và bài tập vận dụng

1. Hoàn thiện ứng dụng với chức năng xoá công việc.
2. Thêm bộ lọc Tất cả, Đang làm, Đã xong.
3. Thêm đếm số công việc chưa hoàn thành.

# Dự án: To-do List — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

## Tình huống, ví dụ và tự kiểm tra

**Yêu cầu:** thêm việc không rỗng, đánh dấu hoàn thành, xóa theo id và hiển thị số việc còn lại. Id dùng để nhận diện ổn định, không dùng vị trí mảng khi có xóa/sắp xếp.

```
const tasks = [
  {id: "a", title: "Đọc bài", done: false},
  {id: "b", title: "Thực hành", done: true}
];
const next = tasks.map(t => t.id === "a" ? {...t, done: true} : t);
console.assert(tasks[0].done === false);
console.assert(next.filter(t => !t.done).length === 0);
```

**Tự luyện:** lưu localStorage, tải lại và xử lý JSON hỏng. **Hướng giải:** dùng try/catch khi đọc, xác thực cấu trúc mảng; lưu trạng thái thay vì HTML sinh ra. Thử tiêu đề chứa dấu < và > để chắc chúng hiển thị như chữ, không trở thành thẻ.

## Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

CHƯƠNG 3 · BÀI 1

# 3.1. Arrow function & Destructuring

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

### Mục tiêu bài học

- Viết gọn code bằng arrow function
- Dùng destructuring để tách giá trị từ đối tượng và mảng
- Sử dụng toán tử spread và template string

## 1. Arrow function

Arrow function là cú pháp ngắn gọn cho hàm, đặc biệt hữu ích với callback của các phương thức mảng như `map`, `filter`, `reduce`.

```
const so = [1, 2, 3, 4, 5];
const binhPhuong = so.map((x) => x * x);   // [1, 4, 9, 16, 25]
const chan = so.filter((x) => x % 2 === 0); // [2, 4]
const tong = so.reduce((t, x) => t + x, 0); // 15
```

## 2. Destructuring

Destructuring cho phép tách nhanh giá trị từ đối tượng hoặc mảng ra biến riêng. Với đối tượng, tên biến trùng tên thuộc tính; với mảng, tách theo vị trí.

```
const sv = { ten: "An", tuoi: 20, lop: "CNTT" };
const { ten, tuoi } = sv;

const [a, b] = [10, 20];
function hienThi({ ten, lop = "Chưa có" }) {
  console.log(ten, lop);
}
```

## 3. Spread và rest

Toán tử spread `...` trải các phần tử của mảng hoặc đối tượng ra; dùng để sao chép hoặc gộp dữ liệu mà không làm thay đổi dữ liệu gốc.

```
const m1 = [1, 2];
const m2 = [...m1, 3, 4];               // [1, 2, 3, 4]
const sv2 = { ...sv, tuoi: 21 };        // sao chép và sửa tuổi
```

## 4. Template string

Chuỗi đặt trong dấu backtick cho phép chèn biểu thức bằng `${}` và viết nhiều dòng: ` Xin chào ${ten}, bạn ${tuoi} tuổi `.

## Tóm tắt

- Arrow function giúp callback ngắn gọn.
- Destructuring tách giá trị từ đối tượng và mảng; spread sao chép hoặc gộp dữ liệu.
- Template string dùng backtick và `${}`.

## Câu hỏi ôn tập và bài tập vận dụng

1. Cho mảng điểm, dùng filter và map để lấy các điểm từ 5 trở lên rồi tăng 0.5 điểm.
2. Dùng destructuring lấy tên và lớp từ đối tượng sinh viên.
3. Dùng spread tạo bản sao của một đối tượng và thay đổi một thuộc tính.

# Arrow function & Destructuring — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

## Tình huống, ví dụ và tự kiểm tra

**Tình huống:** arrow function có dấu ngoặc nhọn nhưng không return nên cho undefined. Arrow viết một biểu thức có thể trả ngầm; trả object cần bọc bằng ngoặc tròn.

```
const toLabel = ({id, title = "Chưa có tên"}) => ({id, label: title});
const item = toLabel({id: 1});
console.assert(item.label === "Chưa có tên");
const [first, ...rest] = [1, 2, 3];
console.assert(first === 1 && rest.length === 2);
```

Arrow không tạo this riêng. Đừng đổi mọi method sang arrow chỉ vì cú pháp ngắn; this cần hiểu theo nơi định nghĩa hoặc cách gọi. Spread object/mảng là bản sao nông, không sao chép sâu mọi đối tượng lồng.

**Tự luyện:** đổi tên thuộc tính khi destructuring và đặt giá trị mặc định. **Hướng giải:** giá trị mặc định được dùng cho undefined, không tự thay thế null.

## Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

CHƯƠNG 3 · BÀI 2

# 3.2. Promise & Async/Await

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu xử lý bất đồng bộ và callback
> - Sử dụng Promise với then, catch
> - Viết mã bất đồng bộ dễ đọc bằng async/await

## 1. Bất đồng bộ

Một số tác vụ tốn thời gian như gọi API, đọc file, hẹn giờ không nên chặn chương trình. JavaScript xử lý chúng bất đồng bộ: tác vụ chạy nền và báo kết quả khi xong. Cách cũ là dùng callback, nhưng lồng nhiều callback gây khó đọc (callback hell).

## 2. Promise

Promise là đối tượng đại diện cho một tác vụ bất đồng bộ. Promise có ba trạng thái: pending (đang chờ), fulfilled (hoàn thành thành công) và rejected (thất bại). Sau khi hoàn thành thành công, Promise ở trạng thái fulfilled và trả kết quả qua `.then()`; lỗi được bắt bằng `.catch()`.

```
const cho = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

cho(1000)
  .then(() => console.log("Sau 1 giây"))
  .catch((loi) => console.error(loi));
```

## 3. async và await

Cú pháp `async`/`await` giúp viết mã bất đồng bộ trông giống mã đồng bộ: từ khoá `await` tạm dừng hàm `async` cho tới khi Promise được giải quyết. Dùng `try...catch` để bắt lỗi.

```
async function taiDuLieu() {
  try {
    await cho(500);
    console.log("Đã tải xong");
  } catch (loi) {
```

```
    console.error("Lỗi:", loi);
  }
}
```

## 4. Chạy song song

`Promise.all([p1, p2])` chờ tất cả Promise hoàn thành và trả về mảng kết quả; nếu một Promise thất bại thì toàn bộ thất bại. Dùng khi các tác vụ độc lập với nhau để tiết kiệm thời gian.

## Tóm tắt

- Bất đồng bộ giúp chương trình không bị chặn khi chờ tác vụ chậm.
- Promise có ba trạng thái: pending, fulfilled, rejected.
- `async`/`await` kèm `try...catch` cho mã dễ đọc; `Promise.all` chạy song song.

## Câu hỏi ôn tập và bài tập vận dụng

1. Viết hàm `cho(ms)` trả về Promise và dùng async/await để in ba dòng cách nhau 1 giây.
2. Dùng `Promise.all` chờ hai tác vụ giả lập cùng lúc và in tổng thời gian.
3. Giải thích sự khác nhau giữa `.then()` và `await`.

# Promise & Async/Await — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

## Tình huống, ví dụ và tự kiểm tra

**Hỏi thực tế:** "promsie là gì, then catch để làm gì?" Promise biểu diễn kết quả bất đồng bộ: pending, fulfilled hoặc rejected. then xử lý kết quả, catch xử lý lỗi; async trả Promise, await chờ bên trong hàm async.

```
async function readValue() {
  const value = await Promise.resolve(5);
  return value * 2;
}
readValue().then(value => console.assert(value === 10));
```

Không đọc biến kết quả ngay sau khi khởi động request rồi cho rằng dữ liệu đã về. Với nhiều việc độc lập, Promise.all cho phép chờ chung nhưng sẽ reject khi một Promise reject; chọn cách xử lý theo nghiệp vụ, không nuốt lỗi bằng mảng rỗng tùy tiện.

**Tự luyện:** giả lập thao tác thất bại và bảo đảm nút được mở lại. **Hướng giải:** dùng try/catch/finally, phân biệt đang tải, thành công, rỗng và lỗi trong giao diện.

## Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

CHƯƠNG 3 · BÀI 3

# 3.3. Module & Fetch API

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Chia mã thành các module bằng import và export
> - Gọi API bằng fetch và xử lý dữ liệu JSON
> - Xử lý lỗi và trạng thái khi gọi API

## 1. Module

Module cho phép chia chương trình thành nhiều file nhỏ. Dùng `export` để chia sẻ hàm, biến ra ngoài và `import` để sử dụng. Khi dùng trên trình duyệt phải khai báo `type="module"` ở thẻ script.

```
// toan.js
export function cong(a, b) { return a + b; }
export default function nhan(a, b) { return a * b; }

// main.js
import nhan, { cong } from "./toan.js";
```

## 2. Fetch API

Hàm `fetch(url)` gửi yêu cầu HTTP và trả về Promise. Kết quả là đối tượng Response; gọi `.json()` để đọc nội dung JSON. Chú ý `fetch` chỉ báo lỗi khi mất kết nối, còn mã trạng thái 404 hoặc 500 vẫn là thành công về mặt mạng nên cần kiểm tra `res.ok`.

```
async function layKhoaHoc() {
  const res = await fetch("/api/courses");
  if (!res.ok) throw new Error("Lỗi " + res.status);
  return res.json();
}
```

## 3. Gửi dữ liệu

Để gửi dữ liệu lên máy chủ, đặt `method: "POST"`, khai báo header `Content-Type: application/json` và truyền nội dung bằng `body: JSON.stringify(dulieu)`.

```
await fetch("/api/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email, password }),
});
```

## 4. Xử lý lỗi và trạng thái

Khi gọi API nên quản lý ba trạng thái: đang tải, thành công và lỗi; bọc lời gọi trong `try...catch` và hiển thị thông báo thân thiện cho người dùng.

## Tóm tắt

- `export` / `import` chia mã thành module; thẻ script cần `type="module"`.
- `fetch` trả về Promise; cần kiểm tra `res.ok` và gọi `.json()`.
- Gửi dữ liệu dùng `POST` với header `Content-Type` và `body` dạng JSON.

## Câu hỏi ôn tập và bài tập vận dụng

1. Tách các hàm tính toán ra một module riêng và import vào file chính.
2. Gọi một API công khai bằng fetch và hiển thị danh sách kết quả lên trang.
3. Thêm hiển thị trạng thái đang tải và thông báo khi gọi API thất bại.

CHƯƠNG HỌC THÊM · BÀI TỔNG HỢP

# Học thêm.Tổng hợp. Học thêm, bài tập và hướng dẫn giải

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

## JavaScript — Học thêm về dữ liệu, DOM và bất đồng bộ

Tài liệu bổ sung cho khóa JavaScript từ nền tảng tới Fetch API. Ví dụ chạy trong trình duyệt; phần module nên chạy qua máy chủ phát triển thay vì mở file trực tiếp.

> ### Mục tiêu học thêm
>
> - Phân biệt chuyển kiểu và so sánh nghiêm ngặt
> - Xây dựng DOM từ dữ liệu thay vì dùng HTML không tin cậy
> - Xử lý lỗi HTTP và tình huống phản hồi API đến sai thứ tự

## 1. Chuyển kiểu và tham chiếu

Chuỗi nhập từ form vẫn là chuỗi. Number('') trả 0 nên phải kiểm tra chuỗi trắng trước khi chuyển. Sau chuyển, dùng Number.isFinite để loại NaN và vô cực khi yêu cầu một số hữu hạn.

```
function readScore(raw) {
  if (raw.trim() === '') throw new Error('Chưa nhập điểm');
  const score = Number(raw);
  if (!Number.isFinite(score) || score < 0 || score > 10) {
    throw new Error('Điểm không hợp lệ');
  }
  return score;
}
console.log(readScore('7.5')); // 7.5
console.log('5' === 5); // false
```

const ngăn gán lại tên biến, không đóng băng đối tượng. Một mảng khai báo const vẫn có thể push. Spread [...items] tạo mảng mới nhưng chỉ sao chép nông; đối tượng bên trong có thể vẫn dùng chung.

## 2. DOM và sự kiện

Với To-do List, dữ liệu là nguồn chính; DOM là kết quả hiển thị. Mỗi công việc cần mã ổn định để xóa đúng dù danh sách bị sắp xếp. Dùng textContent cho nội dung người dùng nhập, tránh chèn thẳng vào innerHTML.

```
function renderTask(task) {
  const li = document.createElement('li');
  const label = document.createElement('span');
  label.textContent = task.title;
  const button = document.createElement('button');
  button.type = 'button';
  button.textContent = 'Xóa';
  button.addEventListener('click', () => removeTask(task.id));
  li.append(label, button);
  return li;
}
```

removeTask là hàm của ứng dụng: lọc mảng theo id, lưu lại rồi render. Khi xử lý submit, gọi preventDefault nếu form được quản lý bằng JavaScript; kiểm tra title.trim() trước khi thêm. Không gắn lại cùng một listener mỗi lần render trên cùng nút.

## 3. Bất đồng bộ và thứ tự thực thi

await tạm dừng phần tiếp theo của hàm async, không khóa toàn bộ trang. Promise thất bại cần catch hoặc try/catch. Khi hai tác vụ độc lập có thể chạy cùng lúc, Promise.all giảm thời gian chờ nhưng sẽ reject nếu một promise reject.

```
console.log('A');
Promise.resolve().then(() => console.log('B'));
console.log('C');
// A, C, B
```

Trong ví dụ, callback then được chạy sau phần đồng bộ hiện tại. Hiểu thứ tự này giúp tránh đọc dữ liệu trước khi request hoàn tất.

## 4. Fetch và phản hồi cũ

fetch không tự ném lỗi chỉ vì server trả 404 hoặc 500. Cần kiểm tra response.ok trước khi đọc dữ liệu dùng cho giao diện.

```
let controller;
async function searchCourses(term) {
  controller?.abort();
  controller = new AbortController();
  const signal = controller.signal;
  try {
    const response = await fetch('/api/courses?q=' + encodeURIComponent(term), { signal
});
    if (!response.ok) throw new Error('HTTP ' + response.status);
    const data = await response.json();
    if (!signal.aborted) console.log(data);
  } catch (error) {
    if (error.name !== 'AbortError') console.error(error.message);
  }
}
```

Đường dẫn API là ví dụ, cần thay theo backend đang dùng. Hủy request cũ giúp kết quả tìm kiếm trước không ghi đè kết quả sau. Ứng dụng thực tế nên hiện trạng thái tải, rỗng và lỗi thay vì chỉ console.log.

## Câu hỏi ôn tập và bài tập vận dụng

1. Dễ: dự đoán typeof của dữ liệu lấy từ input; giải thích vì sao '2' + '3' là '23'. Đáp án: dữ liệu là chuỗi, toán tử cộng nối chuỗi khi các toán hạng là chuỗi.
2. Trung bình: thêm lọc việc đã hoàn thành vào To-do List nhưng không mất dữ liệu gốc. Giữ toàn bộ tasks, tạo visibleTasks bằng filter theo trạng thái; không thay tasks bằng danh sách đã lọc.
3. Khó: tìm kiếm khi người dùng nhập liên tiếp ba từ khóa, phản hồi lần đầu về cuối. Dùng AbortController hoặc số phiên request để chỉ nhận kết quả mới nhất; thử mạng chậm và lỗi 500.

## Dự án mở rộng và tiêu chí hoàn thành

Thêm lưu localStorage cho To-do List. Khi đọc JSON, bắt lỗi parse và kiểm tra dữ liệu có đúng mảng công việc không. Thử tiêu đề rỗng, chuỗi chứa thẻ HTML, mã trùng và tải lại trang. Bài đạt khi thao tác thêm/xóa/lọc nhất quán, nội dung người dùng được hiển thị như văn bản và lỗi lưu trữ không khiến trang trắng.

## Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects

# Module & Fetch API — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

## Tình huống, ví dụ và tự kiểm tra

**Tình huống:** fetch trả 404 nhưng catch không chạy. fetch không tự coi HTTP 4xx/5xx là Promise rejection; phải kiểm tra response.ok. JSON lỗi cũng cần được xử lý.

```
export async function getCourses(url, signal) {
  const response = await fetch(url, {signal});
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const body = await response.json();
  if (!Array.isArray(body.data)) throw new Error("Sai cấu trúc dữ liệu");
  return body.data;
}
```

**Tự luyện:** thử 200 có dữ liệu, 200 rỗng, 404, JSON hỏng và hủy request. **Hướng giải:** mỗi trạng thái có cách hiển thị rõ; AbortController không đồng nghĩa backend hoàn tác hành động đã nhận. Module trình duyệt cần `type="module"`; lỗi CORS phải sửa cấu hình đúng phía server, không tắt bảo vệ trình duyệt.

## Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide